Electronics & Electrical Communications Department

Introduction to Communications - EECG232

Software Radio
Random Process Analysis

By

Team 11

Submitted to

Dr. Mohamed Nafi

April 26, 2026

Contents

1 Introduction 1
2 Control Flags 1
3 Generation of data 2
3.1 Unipolar Signaling 2
3.2 Polar NRZ Signaling 2
3.3 Polar RZ Signaling 2
3.4 Ensemble Generation 3
3.5 visualizing waveforms 5
4 Analysis 5
4.1 The Statistical Mean 5
4.2 The Ensemble Autocorrelation Function 6
4.3 The Time Mean and Autocorrelation of One Waveform 8
4.4 Bandwidth of the Transmitted Signal 9
5 Full MATLAB code 10

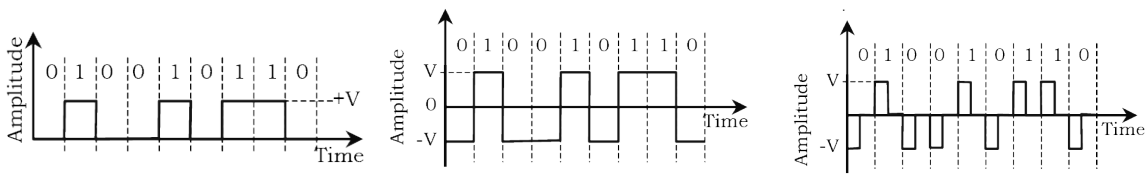
1 Introduction

Software Radio is a technique wherein digital signal processing is used to perform radio communication functions traditionally implemented in analog hardware. This approach allows for greater flexibility and adaptability in communication systems, enabling the implementation of various modulation schemes, error correction techniques, and signal processing algorithms through software updates rather than hardware changes.

This project focuses on generating line encodings for randomly transmitted bits and analyzing the statistical properties of the resulting waveforms. The statistical analysis serves a practical engineering purpose as an ideal transmitted signal should form a stationary and ergodic random process.

Stationarity ensures that the signal's spectral density (PSD) is stable over time, enabling the definition of a fixed power spectral density and predictable bandwidth occupancy. Ergodicity ensures that a single observed waveform is statistically representative of the entire ensemble, which is critical in practice since it can greatly reduce costs by requiring fewer testing procedures during design validation and characterization. Together these properties form the theoretical foundation for receiver filter design, bandwidth estimation, and BER analysis.

This project explores 3 different encoding techniques. Figure 1 shows methods for representing the same binary data in a discrete method. The properties of each will be explored.



a) Unipolar Signaling b) Polar NRZ Signaling c) Polar RZ Signaling

Figure 1: Different line encodings for the same bit sequence

2 Control Flags

To easily configure and manage the entire simulation from a single centralized location, the following control parameters are defined at the very beginning of the script. They govern the data generation process: the signal amplitude (`A = 4`), the resolution of each bit (`sample_per_bit = 8`), and the physical time duration of a single sample (`sample_period = 1e-3`). Furthermore, the dimensions of the random process are defined by the length of each individual waveform (`n_bits = 100`) and the total number of simulated realizations within the ensemble (`n_waveforms = 500`).

```
% Control Flags (Defaults)
control_flags.A = 4;
control_flags.sample_per_bit = 8;
control_flags.sample_period = 1e-3;
control_flags.n_bits = 100;
control_flags.n_waveforms = 500;
```

3 Generation of data

The following MATLAB code generates a random bit stream of 100 bits:

```
random_bit_stream = randi([0 1], 1, 100);
```

The `randi([0 1], x, y)` function returns an $x \times y$ array of uniformly distributed numbers between 0 and 1.

3.1 Unipolar Signaling

For unipolar signaling, the bits `[0 1]` map to `[0 A]` to get the logic levels. The Repeat Matrix function `repmat(levels, spb, 1)` duplicates the $1 \times$ `n_bits` matrix (`spb`) times vertically creating a `spb` $\times$ `n_bit` matrix. The `reshape()` function transforms the matrix into a $1 \times$(`spb` $\cdot$ `n_bits` list along the columns:

A visualized exmaple if `n_bit = 2` and `spb = 2`

$$\text{bits: } [0\ 1] \xrightarrow{\times \mathrm{A}} [0\ \mathrm{A}] \xrightarrow{\text{repmat}} \begin{bmatrix} 0 & \mathrm{A} \\ 0 & \mathrm{A} \end{bmatrix} \xrightarrow{\text{reshape}} [0\ 0\ \mathrm{A}\ \mathrm{A}]$$

```
function waveform = unipolar_nrz(bits, A, spb)
  levels = bits * A;
  waveform = reshape(repmat(levels(:)', spb, 1), 1, []);
end
```

3.2 Polar NRZ Signaling

The mapping for polar non-return-to-zero maps `[0 1]` into `[-A A]` through the following transformations:

$$\text{bits: } [0\ 1] \xrightarrow{\times 2} [0\ 2] \xrightarrow{-1} [-1\ 1] \xrightarrow{\times \mathrm{A}} [-\mathrm{A}\ \mathrm{A}]$$

MATLAB/Octave code for Polar NRZ:

```
function waveform = unipolar_nrz(bits, A, spb)
  levels = bits * A;
  waveform = reshape(repmat(levels(:)', spb, 1), 1, []);
end
```

3.3 Polar RZ Signaling

For polar return-to-zero, we first initialize the waveform matrix size and only set half of the `spb` to `A` and `-A` for the high and low bits respectively:

A visualized exmaple if `n_bit = 2` and `spb = 4`

$$[0\ 1] \xrightarrow{\times 2} [0\ 2] \xrightarrow{-1} [-1\ 1] \xrightarrow{\times \mathrm{A}} [-\mathrm{A}\ \mathrm{A}] \xrightarrow{\text{repmat}} \begin{bmatrix} -\mathrm{A} & \mathrm{A} \\ -\mathrm{A} & \mathrm{A} \\ -\mathrm{A} & \mathrm{A} \\ -\mathrm{A} & \mathrm{A} \end{bmatrix} \xrightarrow{\text{Zero}} \begin{bmatrix} -\mathrm{A} & \mathrm{A} \\ -\mathrm{A} & \mathrm{A} \\ 0 & 0 \\ 0 & 0 \end{bmatrix} \xrightarrow{\text{reshape}} [-\mathrm{A}\ -\mathrm{A}\ 0\ 0\ \mathrm{A}\ \mathrm{A}\ 0\ 0]$$

```
1  function waveform = polar_rz(bits, A, spb)
2    half_spb = floor(spb / 2);
3    levels = (2*bits(:)' - 1) * A;
4    raw_matrix = repmat(levels, spb, 1);
5
6    % Zero out the second half of each bit period
7    raw_matrix(half_spb+1:end, :) = 0;
8
9    waveform = reshape(raw_matrix, 1, []);
10 end
```

3.4 Ensemble Generation

The Random Process is characterized by randomized bits and a phase shift implemented as a circular shift of a random integer between 0 and `spb-1` as shown in Lines 7,8 below.

For a given random process $x(t)$, an experiment generates a **Realization** for the process. Figure 2 showcases an interactive polar nrz random process which can be tried Here. Clicking the shuffle button generates new realizations.

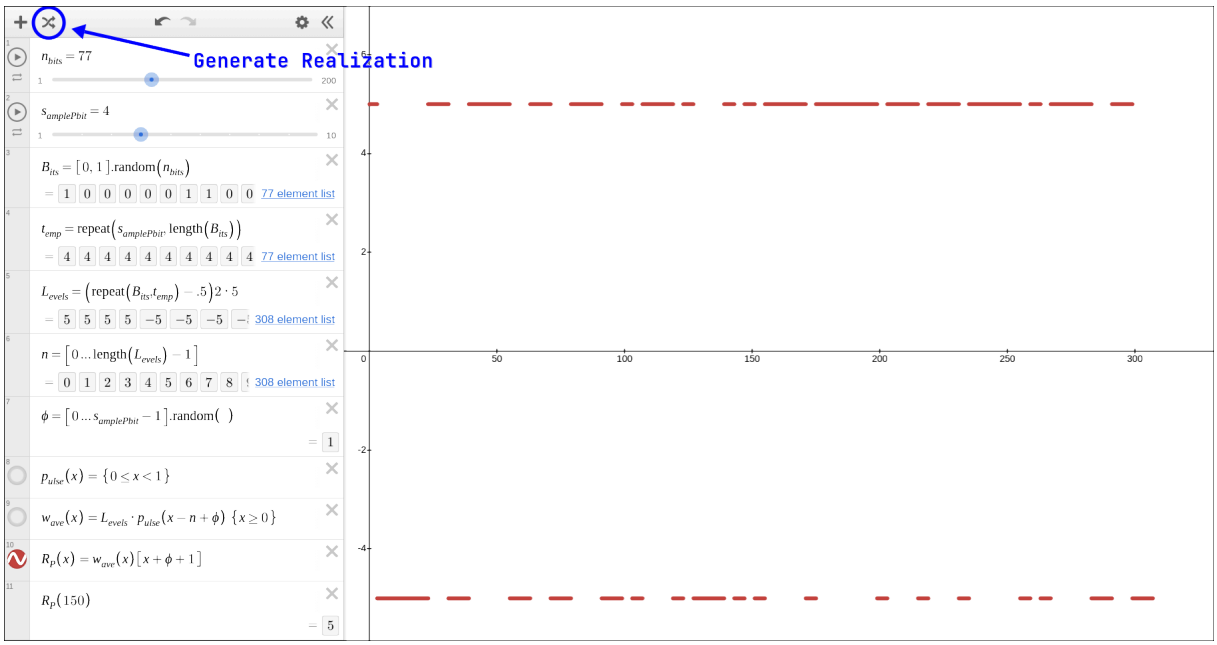


Figure 2: Polar NRZ Random Process.

Multiple of such realizations can be simulated to construct an **Ensemble** of random variables that can be used to calculate the statistical properties of the random process.

`generate_ensemble()` acts as the core engine for generating a random process, using a flexible input system where `nargin` and `isempty` checks and prioritize user-provided arguments over the default values stored in `control_flags`.

To ensure high performance, it pre-allocates the ensemble matrix with `zeros(n_waveforms, waveform_length)`.

Inside the for loop, it generates a unique `random_bit_stream` for every realization and passes it to the `line_code_func` handle, allowing it to produce waveforms based on the desired linecode.

To model a WSS random process, the code calculates a phase variable ranging from `0` to `spb-1`. It then applies this offset using circshift`(raw_wf, [0, -phase])`, which ensures that the bit transitions do not always align perfectly with the start of the sample buffer. Each shifted raw_wf is then stored as a row in the final matrix, creating a comprehensive ensemble ready for statistical analysis.

```
function ensemble = generate_ensemble(line_code_func, control_flags,
n_waveforms, n_bits, A, spb)
  % Use provided arguments or fall back to control_flags struct
  if nargin < 6 || isempty(spb), spb = control_flags.sample_per_bit;
end
  if nargin < 5 || isempty(A), A = control_flags.A; end
  if nargin < 4 || isempty(n_bits), n_bits = control_flags.n_bits; end
  if nargin < 3 || isempty(n_waveforms), n_waveforms =
control_flags.n_waveforms; end

  % Direct allocation for memory efficiency
  waveform_length = n_bits * spb;
  ensemble = zeros(n_waveforms, waveform_length);

  for i = 1:n_waveforms
      random_bit_stream = randi([0 1], 1, n_bits);
      raw_wf = line_code_func(random_bit_stream, A, spb);

      % Apply random phase by circular shift and store directly in
matrix
      phase = randi([0, spb-1]);
      ensemble(i, :) = circshift(raw_wf, [0, -phase]);
  end
end
```

To make things cleaner, a function called `generate_all_ensembles()` is implemented to return a cell array containing 3 ensembles, one for each line code.

```
function ensembles = generate_all_ensembles(control_flags, n_waveforms,
n_bits, A, spb)
    % Pass along optional arguments (empty brackets will trigger
defaults in generate_ensemble)
    if nargin < 5, spb = []; end
    if nargin < 4, A = []; end
    if nargin < 3, n_bits = []; end
    if nargin < 2, n_waveforms = []; end

    % Generates ensembles in order: Unipolar, Polar NRZ, Polar RZ
    ens_uz = generate_ensemble(@unipolar_nrz, control_flags,
n_waveforms, n_bits, A, spb);
```

```
    ens_pz = generate_ensemble(@polar_nrz, control_flags, n_waveforms,
n_bits, A, spb);
    ens_rz = generate_ensemble(@polar_rz, control_flags, n_waveforms,
n_bits, A, spb);

    ensembles = {ens_uz, ens_pz, ens_rz};
end
```

3.5 visualizing waveforms

A function `plot_sample_waveforms()` called that takes the all_ensembles and generates a plot of the first 5 waveforms of each ensemble was implemented but it's not shown here as it's has nothing to do with the core logic of the simulation.

The function is used in the following code to plot the first 5 wavforms of each ensemble

```

%% Generate Ensembles
ensembles_5h = generate_all_ensembles(control_flags);

%% Plot waveforms
plot_sample_waveforms(ensembles_5h, control_flags);

```

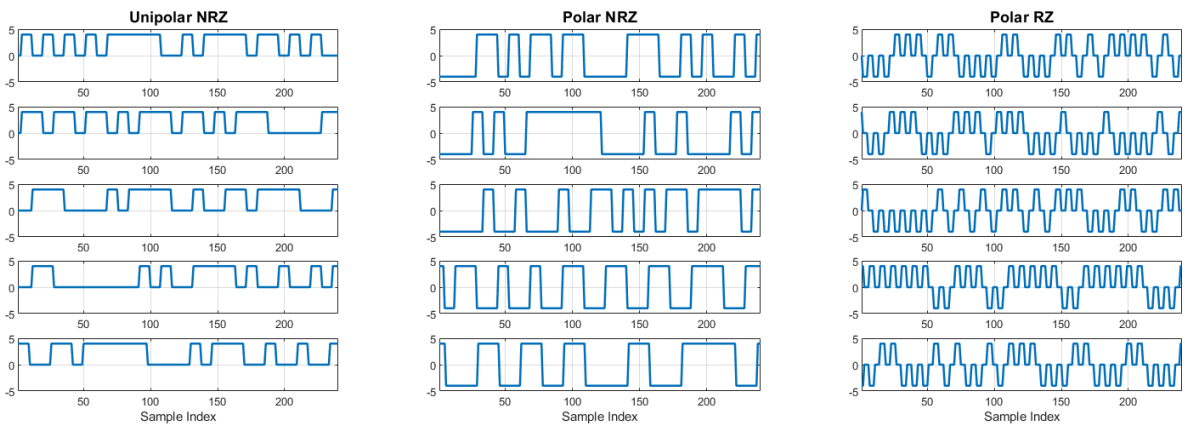


Figure 3: Realizations for each type of line encoding.

The waveforms in Figure 3 were generated using the above code, showing a sample of the structure of the three line codes.

4 Analysis

The following analysis is performed on an ensemble of $N = 500$ waveforms, each containing $L = 100$ bits at spb $= 8$ samples per bit, giving a total waveform length of $L_s = 800$ samples. The amplitude is $A = 4$.

4.1 The Statistical Mean

The statistical mean is the expected value of the random process at each time instant t, averaged across all N realizations:

$$\mu_x(t) = E[x(t)] = \frac{1}{N}\sum_{i=1}^{N} x_{i(t)}$$

This yields a 1×800 vector — one mean value per time sample. Figure 4 shows how the mean is nearly constant across time with a smaller variance over a larger ensemble.

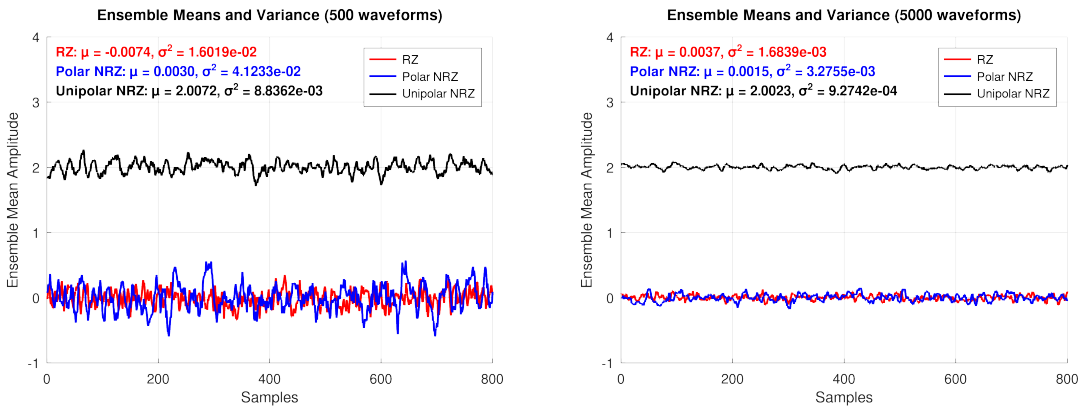


Figure 4: The statistical mean of each line-encoding for 500 and 5k waveforms.

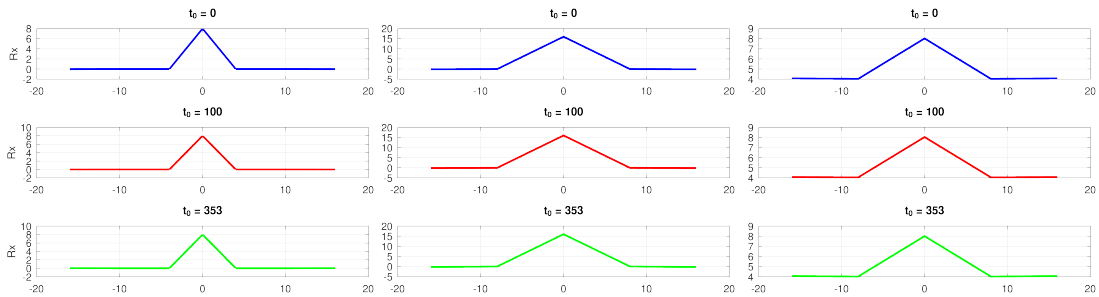
```
function ensembles = generate_all_ensembles(n_waveforms=500)
 [ens_rz, wfs_rz] = generate_ensemble(@rz, n_waveforms);
 [ens_pz, wfs_pz] = generate_ensemble(@polar_nrz, n_waveforms);
 [ens_uz, wfs_uz] = generate_ensemble(@unipolar_nrz, n_waveforms);
  ensembles = {ens_rz, ens_pz, ens_uz};
endfunction
function plot_means(ensembles)
  names = {'RZ', 'Polar NRZ', 'Unipolar NRZ'};
  colors = {'r', 'b', 'k'};
  n_waveforms = size(ensembles{1}, 1);
  figure;
  hold on;
  for i = 1:length(ensembles)
      ens_mean = mean(ensembles{i}, 1);   %1 x 800 array
      average = mean(ens_mean); variance = var(ens_mean);
      plot(ens_mean, 'Color', colors{i}, 'DisplayName', names{i});
  end
endfunction
ens_5h = generate_all_ensembles(500);
plot_means(ens_5h)
ens_5k = generate_all_ensembles(5000);
plot_means(ens_5k)
```

4.2 The Ensemble Autocorrelation Function

The ensemble autocorrelation function says how much a certain reading influences the next one separated by a lag τ:

$$R_x(t,\tau) = E[x(t)\cdot x(t+\tau)] = \frac{1}{N}\sum_{i=1}^{N}\frac{1}{L_s-\tau}\sum_{t=1}^{L_s-\tau} x_i(t)\cdot x_i(t+\tau)$$

In Figure 5, the autocorrelation function is calculated for varying start times over varying lags. It can be seen that $R_x(t,\tau)$ doesn't depend on the initial time and only depends on the time difference ($R_x(t,\tau) = R_x(\tau)$). Additionally, since μ_x was calculated to be a constant over time, the random process is **Wide Sense Stationary (WSS)**. That can also be verified by observing how the autocorrelation is an even function.



a) Polar RZ Autocorrelation ($R_x(0) = 8$, Roll-off= 0@4)

b) Polar NRZ Autocorrelation ($R_x(0) = 16$, Roll-off= 0@8)

c) Unipolar NRZ Autocorrelation ($R_x(0) = 8$, Roll-off= 4@8)

Figure 5: Statistical Autocorrelation of each line-encoding for different start times.

```
function rx = autocorr_from_t(ensemble, max_lag, t_start=1)
  sub = ensemble(:, t_start:end);
  [n_waveforms, n_samples] = size(sub);
  rx = zeros(1, 2*max_lag+1);
  for k = -max_lag:max_lag
    if k>=0
      product = sub(:,1:n_samples-k) .* sub(:,k+1:n_samples); %NxLs
    else
      product = sub(:,-k+1:n_samples) .* sub(:,1:n_samples+k); %NxLs
    end
    average_per_waveform = mean(product, 2); %Nx1
    rx(k+max_lag+1) = mean(average_per_waveform);  %1x1
  end
endfunction
function plot_autocorrs(ensemble, max_lag=16, t1=1, t2=101, t3=354)
  rx1 = autocorr_from_t(ensemble, max_lag, t1);
  rx2 = autocorr_from_t(ensemble, max_lag, t2);
  rx3 = autocorr_from_t(ensemble, max_lag, t3);
  lags = -max_lag:max_lag;
  figure;
  subplot(3,1,1); plot(lags, rx1, 'b', 'LineWidth', 2);
  ylabel('Rx'); title(sprintf('t_0 = %d', t1-1)); grid on;
  subplot(3,1,2); plot(lags, rx2, 'r', 'LineWidth', 2);
  ylabel('Rx'); title(sprintf('t_0 = %d', t2-1)); grid on;
  subplot(3,1,3); plot(lags, rx3, 'g', 'LineWidth', 2);
  ylabel('Rx'); title(sprintf('t_0 = %d', t3-1)); grid on;;
endfunction
plot_autocorrs(ens_5h{1})
plot_autocorrs(ens_5h{2})
plot_autocorrs(ens_5h{3})
```

4.3 The Time Mean and Autocorrelation of One Waveform

For a single realization $x_i(t)$, the time mean and autocorrelation both operate on a single 1×800 row of the ensemble matrix, producing a scalar mean and a $1 \times (\tau_{\max} + 1)$ autocorrelation vector respectively as shows in Figure 6.

$$\langle x(t) \rangle = \overline{x}_i = \frac{1}{L_s} \sum_{t=1}^{L_s} x_i(t)$$

$$\langle x(t) \cdot x(t+\tau) \rangle = \frac{1}{L_s - \tau} \sum_{t=1}^{L_s - \tau} x_i(t) \cdot x_i(t+\tau)$$

While similar, the time-based autocorrelation differs from the ensemble R_x in Figure 5.

However, increasing the number of bits makes it converge to the ensemble R_x. Figure 7 shows the calculation for the mean and autocorrelation of a waveform generated with 100 kilobits.

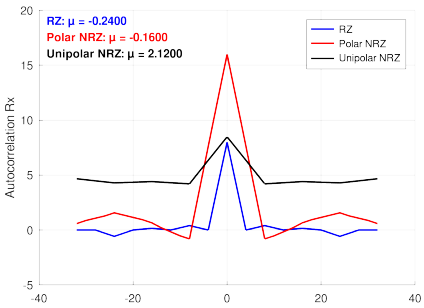


Figure 6: Mean and autocorrelation of one [100-bit] waveform.

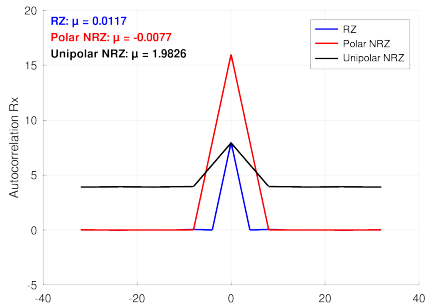


Figure 7: Mean and autocorrelation of a 10^5-bit waveform.

This characterizes the random process as **Ergodic**, meaning that the process statistics can be characterized through a single sufficiently long observation window.

```
function rx_time = time_autocorr(waveform, max_lag=16)
  n = length(waveform);
  rx_time = zeros(1, 2*max_lag + 1);
  for k = -max_lag:max_lag
    if k>=0
    rx_time(k+max_lag+1) = mean(waveform(1:n-k) .* waveform(k+1:n));
    else
    rx_time(k+max_lag+1) = mean(waveform(-k+1:n) .* waveform(1:n+k));
    end
  end
endfunction

hold on; grid on; n = 1e5;
funcs = {@rz, @polar_nrz, @unipolar_nrz}; colors = {'b', 'r', 'k'};
names = {'RZ', 'Polar NRZ', 'Unipolar NRZ'};
for i = 1:3
  wv = ens_5h{i}(1,:);  % Swap this with 17 to generate Figure 6
  wv = generate_ensemble(funcs{i}, 1, n); % Make a 1e5 bit waveform
    plot(-32:32, time_autocorr(wv,32), 'Color', colors{i}, ...
         'LineWidth', 1.5);
```

```
    stats_text = sprintf('%s: \\mu = %.4f', names{i}, mean(wv));
    text(0.02, 0.98 - (i-1)*0.06, stats_text, 'Units', ...
        'normalized', 'Color', colors{i}, 'FontSize', 10, ...
        'FontWeight', 'bold', 'VerticalAlignment', 'top');
end
ylabel('Autocorrelation Rx'); legend(names); hold off;
```

4.4 Bandwidth of the Transmitted Signal

The power spectral density is obtained as the Fourier transform of the autocorrelation function. Applying the FFT gives the PSD in Figure 8, assuming that each sample corresponds to a time window of 10ms.

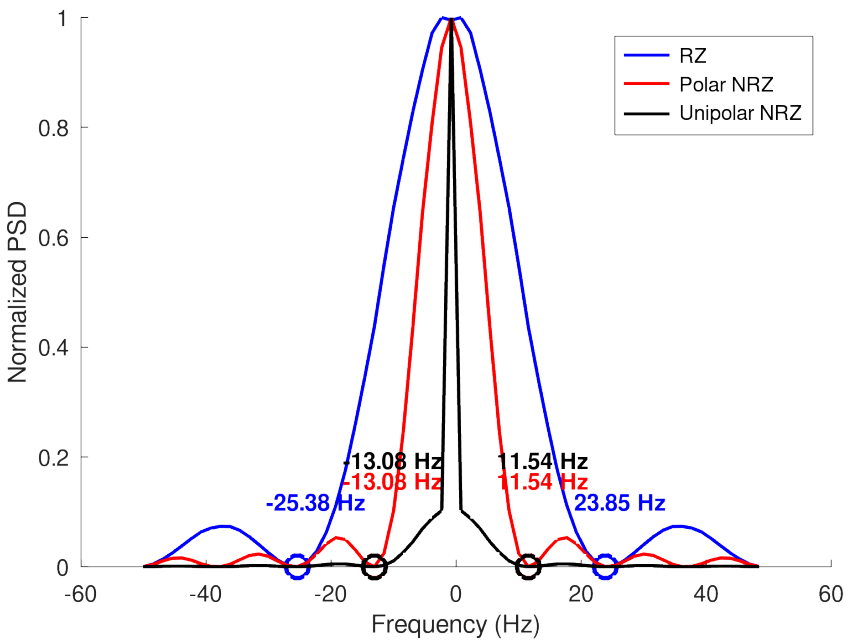


Figure 8: The PSD of each line encoding with the first null marked.

For the bandwidth, we choose the point of first zero of the $S_x(f)$ function giving the baseband bandwidth shown in the figure. The passband bandwidth is given by:

$$\text{Passband BW} = \begin{cases} 23.85 + 25.38 = 49.23\text{Hz for Polar RZ} \\ 11.54 + 13.08 = 24.62\text{Hz for Polar NRZ} \\ 11.54 + 13.08 = 24.62\text{Hz for Unipolar NRZ} \end{cases}$$

Analytically, the $S_x(f)$ is given as a $\text{sinc}^2()$ function for each encoding since it is the transform of a convolved $\text{rect}()$. Accordingly, the first zero is given by $\frac{1}{T_p}$ where $T_p = 80\text{ms}$ is the pulse width. This gives a passband bandwidth of $\frac{2}{T_p} = 25\text{Hz}$ aligning with the NRZ encodings.

The discrepancy in the RZ encoding happens since it only transmits data for half the pulse. Effectively having a T_p of 40ms and double the bandwidth of NRZ encodings.

```
max_lag = 32; N=2*max_lag+1;
freq_axis = ( -0.5: 1/N : 0.5 - 1/N ); %N elements
```

```
freq_axis = freq_axis * 1/10e-3 ; % 1sample -> 10ms
funcs = {@rz, @polar_nrz, @unipolar_nrz}; colors = {'b', 'r', 'k'};
names = {'RZ', 'Polar NRZ', 'Unipolar NRZ'};
for i = 1:3
  rxs{i} = autocorr_from_t(ens_5h{i}, max_lag);
  rxs_shift{i} = ifftshift(rxs{i}); % Centers at lag 0 [0..16 -1..-16]
  PSDs{i} = abs(fft(rxs_shift{i}));
  PSDs{i} = PSDs{i}/max(PSDs{i});   % Normalize the FFT
  PSDs0{i} = fftshift(PSDs{i});     % Centers the 0Hz frequency

  hold on;
  p(i) = plot(freq_axis, PSDs0{i}, 'Color', colors{i},...
              'LineWidth', 1.5);
  ind = find(PSDs0{i} == min(PSDs0{i}));
  labels = cellstr(num2str(freq_axis(ind)', '%.2f Hz'));
  plot(freq_axis(ind), PSDs0{i}(ind), 'ro', 'Color', colors{i},...
       'MarkerSize', 10, 'LineWidth', 2);
  text(-5+freq_axis(ind), i*3.8e-2+0.08+PSDs0{i}(ind), labels,...
      'Color', colors{i}, 'FontSize', 10, 'FontWeight', 'bold');
end
legend(p, names); xlabel('Frequency (Hz)'); ylabel('Normalized PSD');
```

5 Full MATLAB code

```
global A_default samples_per_bit_default
A_default = 4;
samples_per_bit_default = 8;

function waveform = unipolar_nrz(bits, A, spb)
  global A_default samples_per_bit_default
  if nargin < 3, spb = samples_per_bit_default; end
  if nargin < 2, A = A_default; end
  levels = bits * A;
  waveform = reshape(repmat(levels, spb, 1), 1, []);
endfunction

function waveform = polar_nrz(bits, A, spb)
  global A_default samples_per_bit_default
  if nargin < 3, spb = samples_per_bit_default; end
  if nargin < 2, A = A_default; end
  levels = (2*bits - 1) * A;
  waveform = reshape(repmat(levels, spb, 1), 1, []);
endfunction

function waveform = rz(bits, A, spb)
  global A_default samples_per_bit_default
  if nargin < 3, spb = samples_per_bit_default; end
  if nargin < 2, A = A_default; end
  n = length(bits);
  waveform = zeros(1, n * spb);
```

```
  half_spb = floor(spb / 2);
  for i = 0:n-1
    if bits(i+1) == 1
      waveform(i*spb+1 : i*spb+half_spb) = A;
    else
      waveform(i*spb+1 : i*spb+half_spb) = -A;
    end
  end
endfunction

function [ensemble, waveforms] = generate_ensemble(code_func, ...
  n_waveforms=500, n_bits=100, A, spb)
  global A_default samples_per_bit_default
  if nargin < 5, spb = samples_per_bit_default; end
  if nargin < 4, A = A_default; end
  waveforms = cell(n_waveforms, 1);
  for i = 1:n_waveforms
    bits = rand(n_bits, 1) > 0.5; bits = bits(:)';
    waveforms{i} = code_func(bits, A, spb);
    phase = randi([0 spb-1]);
waveforms{i} = [ waveforms{i}(1+phase:end), waveforms{i}(1:phase) ];
  end
  waveform_length = length(waveforms{1});
  ensemble = zeros(n_waveforms, waveform_length);
  for i = 1:n_waveforms
    ensemble(i, :) = waveforms{i};
  end
endfunction
function ensembles = generate_all_ensembles(n_waveforms=500)
 [ens_rz, wfs_rz] = generate_ensemble(@rz, n_waveforms);
 [ens_pz, wfs_pz] = generate_ensemble(@polar_nrz, n_waveforms);
 [ens_uz, wfs_uz] = generate_ensemble(@unipolar_nrz, n_waveforms);
  ensembles = {ens_rz, ens_pz, ens_uz};
endfunction

function plot_waveforms3(code_func, name, n_show=5)
  [ensemble, waveforms] = generate_ensemble(code_func);
  n = min(n_show, length(waveforms));
  figure;
  for i = 1:n
    subplot(n, 1, i);
    plot(waveforms{i}(1:30*8), 'LineWidth', 2);
    ylabel('Amp');
    ylim([-6 6]);
    if i == 1
      title(sprintf('First %d %s Waveforms', n, name));
    end
  end
  xlabel('Sample');
endfunction

```

```
function plot_means(ensembles)
  names = {'RZ', 'Polar NRZ', 'Unipolar NRZ'};
  colors = {'r', 'b', 'k'};
  n_waveforms = size(ensembles{1}, 1);
  figure;
  hold on;
  for i = 1:length(ensembles)
      current_ens = ensembles{i};
      ens_mean = mean(current_ens, 1);   % A 1 x 800 array
      % Calculate the mean of means and variance of means
      average = mean(ens_mean);
      variance = var(ens_mean);
      % Plot the 1 x 800 ensemble mean
      plot(ens_mean, 'Color', colors{i}, 'LineWidth', 1.5, ...
           'DisplayName', names{i});
      stats_text = sprintf('%s: \\mu = %.4f, \\sigma^2 = %.4e', ...
                           names{i}, average, variance);
      text(0.02, 0.98 - (i-1)*0.06, stats_text, 'Units', ...
          'normalized', 'Color', colors{i}, 'FontSize', 10, ...
          'FontWeight', 'bold', ...
          'VerticalAlignment', 'top');
  end
  ylim([-1, 4]);
  xlabel('Samples');
  ylabel('Ensemble Mean Amplitude');
  title(sprintf('Ensemble Means and Variance (%d waveforms)', ...
        n_waveforms));
  legend('Location', 'northeast');
  grid on;
  hold off;
endfunction

function rx = autocorr_from_t(ensemble, max_lag, t_start=1)
  sub = ensemble(:, t_start:end);
  [n_waveforms, n_samples] = size(sub);
  rx = zeros(1, 2*max_lag+1);
  for k = -max_lag:max_lag
    if k>=0
      product = sub(:,1:n_samples-k) .* sub(:,k+1:n_samples); %NxS
    else
      product = sub(:,-k+1:n_samples) .* sub(:,1:n_samples+k); %NxS
    end
    average_per_waveform = mean(product, 2); %Nx1
    rx(k+max_lag+1) = mean(average_per_waveform);  %1x1
  end
endfunction

function plot_autocorrs(ensemble, max_lag=16, t1=1, t2=101, t3=354)
  rx1 = autocorr_from_t(ensemble, max_lag, t1);
  rx2 = autocorr_from_t(ensemble, max_lag, t2);
  rx3 = autocorr_from_t(ensemble, max_lag, t3);
```

```
  lags = -max_lag:max_lag;
  figure;
  subplot(3,1,1); plot(lags, rx1, 'b', 'LineWidth', 2);
  title(sprintf('t_0 = %d', t1-1)); grid on;
  subplot(3,1,2); plot(lags, rx2, 'r', 'LineWidth', 2);
  title(sprintf('t_0 = %d', t2-1)); grid on;
  subplot(3,1,3); plot(lags, rx3, 'g', 'LineWidth', 2);
  title(sprintf('t_0 = %d', t3-1)); grid on;
endfunction

function rx_time = time_autocorr(waveform, max_lag=16)
  n = length(waveform);
  rx_time = zeros(1, 2*max_lag + 1);
  for k = -max_lag:max_lag
    if k>=0
    rx_time(k+max_lag+1) = mean(waveform(1:n-k) .* waveform(k+1:n));
    else
    rx_time(k+max_lag+1) = mean(waveform(-k+1:n) .* waveform(1:n+k));
    end
  end
endfunction

function plot_time_autocorr(n_bits=1e5)
  funcs = {@rz, @polar_nrz, @unipolar_nrz};
  colors = {'b', 'r', 'k'};
  names = {'RZ', 'Polar NRZ', 'Unipolar NRZ'};
  figure; clf; hold on; grid on;
  for i = 1:3
    wv = generate_ensemble(funcs{i}, 1, n_bits);
    % wv = ens_5h{i}(1,:);  % Swap this with 17 to generate Figure 6
    rx = time_autocorr(wv, 32);
    plot(-32:32, rx, 'Color', colors{i}, 'LineWidth', 1.5);
    stats_text = sprintf('%s: \\mu = %.4f', names{i}, mean(wv));
    text(0.02, 0.98 - (i-1)*0.06, stats_text, 'Units', ...
        'normalized', 'Color', colors{i}, 'FontSize', 10, ...
        'FontWeight', 'bold', 'VerticalAlignment', 'top');
  end
  ylabel('Autocorrelation R_x');
  xlabel('Lag (samples)');
  title('Time-Averaged Autocorrelation');
  legend(names);
  hold off;
endfunction

function plot_psd_from_ensembles(ensembles, sample_period=10e-3)
  % Calculates and plots Normalized PSD
  max_lag = 32; N = 2 * max_lag + 1;
  % Frequency axis setup
  freq_axis = (-0.5 : 1/N : 0.5 - 1/N) * (1/sample_period);
  colors = {'b', 'r', 'k'};
  names = {'RZ', 'Polar NRZ', 'Unipolar NRZ'};
```

```
  figure; clf; hold on; grid on;
  for i = 1:3
    rx = autocorr_from_t(ensembles{i}, max_lag);
    % wv = generate_ensemble(funcs{i}, 1, 1e5) ;
    % rx = time_autocorr(wv, max_lag);
    rx_shift = ifftshift(rx);
    psd_raw = abs(fft(rx_shift));
    psd_norm = psd_raw / max(psd_raw);
    psd_plot = fftshift(psd_norm);
p(i) = plot(freq_axis, PSDs0{i}, 'Color', colors{i},'LineWidth',1.5);
    ind = find(PSDs0{i} == min(PSDs0{i}));
    labels = cellstr(num2str(freq_axis(ind)', '%.2f Hz'));
    plot(freq_axis(ind), PSDs0{i}(ind), 'ro', 'Color', colors{i}, ...
          'MarkerSize', 10, 'LineWidth', 2);
    text(-5+freq_axis(ind), i*3.8e-2+0.08+PSDs0{i}(ind), labels, ...
        'Color', colors{i}, 'FontSize', 10, 'FontWeight', 'bold');
  end
  legend(p, names);
  xlabel('Frequency (Hz)'); ylabel('Normalized PSD');
  title('Normalized PSD with bandwidth');
endfunction
```